

# Survival Analysis for Office Occupancy Duration Modelling

---

*A practical guide for data scientists familiar with regression*

---

## Table of Contents

---

1. [Introduction and Motivation](#)
  2. [From Raw Taps to Sessions](#)
  3. [Right-Censoring: The Core Statistical Challenge](#)
  4. [The Survival Function and Hazard Function](#)
  5. [Cox Proportional Hazards Model](#)
  6. [Weibull Accelerated Failure Time Model](#)
  7. [Feature Engineering](#)
  8. [Model Fitting, Likelihood, and AIC](#)
  9. [Model Evaluation and Monitoring](#)
  10. [The Production Pipeline](#)
  11. [Limitations and Extensions](#)
  12. [Quick Reference](#)
  13. [Further Reading](#)
- 

## 1. Introduction and Motivation

---

### The Business Problem

Building operators, facilities managers, and workplace teams need to understand how long people actually spend in the office. This affects desk-booking ratios, HVAC scheduling, cleaning rotas, and capacity planning. Access control systems provide a natural data source: every entry and exit is logged as a timestamped event against a person identifier.

A typical raw event looks like this:

```

person_id, timestamp,          direction
P001,      2025-01-06T08:31:42, in
P001,      2025-01-06T17:08:03, out
P002,      2025-01-06T09:14:55, in
P002,      2025-01-07T09:01:22, in    ← no tap-out before next tap-in!

```

Person P001 has a clean session: 8h 36m. Person P002's session on the 6th has no recorded tap-out — they presumably left through a door held open by a colleague (*tailgating*). This happens in roughly **19–21% of sessions** in real deployments.

## Why Naive Approaches Fail

The obvious fixes all introduce bias:

| Approach                                          | Problem                                                                                                                                                                           |
|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Drop rows without tap-outs                        | Discards ~20% of data; the dropped sessions are not a random sample — longer-stay individuals are more likely to tailgate out, so the remaining data underestimates true duration |
| Impute with dataset mean                          | Treats a 3-hour session and a 10-hour session identically if both are missing a tap-out                                                                                           |
| Cap at<br><code>max_duration_hours=16</code>      | The cap is an upper bound on the censored time, not an estimate; feeding it as a real duration biases the model toward the cap                                                    |
| Fit a linear regression on complete sessions only | Same selection bias as dropping rows, compounded by ignoring uncertainty                                                                                                          |

What we need is a framework that uses the incomplete observations — knowing that someone was still in the building at time  $t^*$  — as genuine evidence, without pretending we know when they left.

## Survival Analysis to the Rescue

Survival analysis was developed precisely for this situation. Originally applied to clinical trial data where some patients are still alive at the study end-date, it provides a principled way to incorporate *censored* observations: cases where we know the event (departure) had not yet occurred, without knowing when it eventually will.

The key insight is that a censored session at  $t = 4h$  *does carry real information: it tells us the person's true duration satisfies  $T \geq 4h$* . Both parametric and semi-parametric models exploit this lower bound via a correctly specified likelihood function (detailed in Section 8).

---

## 2. From Raw Taps to Sessions

---

### The Pairing Algorithm

The raw event log is converted into one row per office session by

`sessions.py:build_sessions()`. The algorithm is simple:

1. Sort each person's events chronologically.
2. When a tap-**in** arrives, open a pending session.
3. When the next event for that person is a tap-**out**, close the session as complete.
4. When the next event is another tap-**in** (before any tap-out), close the pending session as **censored**: the person left without tapping out.
5. If the data ends while a session is still open, close it as censored.

```
# Simplified logic from sessions.py
for _, row in group.iterrows():
    if row["direction"] == "in":
        if pending_in is not None:
            # new in before out → censored
            sessions.append(..., "censored": True,
                           "duration_hours": elapsed_since_pending_in)
            pending_in = row["timestamp"]
        elif row["direction"] == "out" and pending_in is not None:
            sessions.append(..., "censored": False,
                           "duration_hours": elapsed)
            pending_in = None
```

### The Session Schema

The output dataframe has five key columns:

| Column                      | Type            | Description                                                                |
|-----------------------------|-----------------|----------------------------------------------------------------------------|
| <code>person_id</code>      | string          | Identifier for the individual                                              |
| <code>tap_in</code>         | datetime        | Timestamp of entry                                                         |
| <code>tap_out</code>        | datetime or NaT | Timestamp of exit, or missing if censored                                  |
| <code>censored</code>       | bool            | <code>True</code> if tap-out was not observed                              |
| <code>duration_hours</code> | float           | Observed time (complete) or elapsed-to-next-event capped at 16h (censored) |

## The 16-Hour Cap

For censored sessions, `duration_hours` is set to `min(elapsed_to_next_event, max_duration_hours)`. This is **not an estimate** of the true duration. It is a lower-bound placeholder — the model's likelihood function uses it as the point at which observation stopped, not as the true departure time. Changing the cap only affects how long a censored session appears to have lasted before observation ended; it does not change the fact that the true departure is unobserved.

## 3. Right-Censoring: The Core Statistical Challenge

### Defining Censoring

Let  $T$  be the true duration of an office visit. In a complete-data world we observe  $T$  for every visit. In reality, we observe:

- For **complete sessions**: the actual duration  $T = t$
- For **censored sessions**: only that  $T > t$ , where  $t^{**}$  is the censoring time (the elapsed time until the next tap-in for that person, or end of data)

This is called **right-censoring**: the event has not yet occurred by our observation window, so the true value lies to the right of what we saw. Other forms exist:

- **Left-censoring**: the event occurred before observation started (not present here)
- **Interval-censoring**: the event is known to fall within an interval  $(a, b)$  but the exact time is unknown (not present here)

This document deals exclusively with right-censoring.

### The Independence Assumption

A key assumption is that the censoring mechanism is *non-informative*: the probability of tailgating is independent of how long the person would have stayed, given their observable covariates. Stated formally:

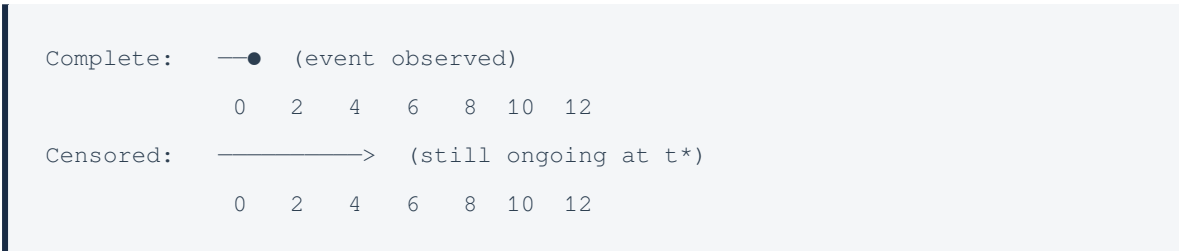
$$T \perp C \mid X$$

where  $C$  is the censoring time and  $X$  are the covariates.

In the office context this is plausible: tailgating is typically a convenience behaviour (following a group through a door) not systematically related to whether someone is planning a two-hour or eight-hour visit. This is different from clinical trials where a patient might drop out *because* they are experiencing side-effects (informative censoring).

### Why Censored Observations Are Not Wasted

Consider a cohort where half the sessions are censored at  $t^* = 6\text{h}$ . The correct mental model is:



The censored session does not tell us the person stayed until 6h. It tells us they stayed at least 6h. This is a hard lower bound that constrains the likelihood. If we used only the 50% complete sessions, we would systematically underestimate durations. The models in this project use both, with the correct likelihood contribution for each type.

### Survival Bias in Practice

In the monitoring data, the mean observed duration for complete sessions is approximately **7.4–7.9h** (varying across batches). The model's predicted mean for all sessions — including the censored ones — is **9.7–10.1h**. This gap is not a model error; it reflects the real phenomenon that people who tailgate out tend to stay longer (they are presumably deeply engaged and leave informally rather than making a dedicated trip to the exit turnstile).

---

## 4. The Survival Function and Hazard Function

---

### The Survival Function $S(t)$

The survival function is defined as:

$$S(t) = P(T > t)$$

It gives the probability that a visit is still ongoing at time  $t$ . Key properties:

- $S(0) = 1$  (everyone is present at the moment of tap-in)
- $S(\infty) = 0$  (everyone eventually leaves)
- $S(t)$  is non-increasing

For a data scientist used to CDFs:  $S(t) = 1 - F(t)$ . Working with  $S$  rather than  $F$  is natural when we care about "how long until the event" rather than "how likely is the event by time  $t$ ".

The non-parametric **Kaplan-Meier estimator** computes  $S(t)$  directly from data:

$$S(t) = \prod_{\{t_i \leq t\}} (1 - d_i / n_i)$$

where  $d_i$  is the number of departures at time  $t_i$  and  $n_i$  is the number of sessions still ongoing just before  $t_i$  (the *risk set*). Censored observations contribute to the denominator  $n_i$  for event times before their censoring point, then drop out — their partial contribution to the denominator is exactly what makes the Kaplan-Meier estimator unbiased under right-censoring. The

`lifelines.KaplanMeierFitter` class computes this; in the production pipeline we use parametric models instead, but Kaplan-Meier is the right first check on new data.

### The Hazard Function $h(t)$

The hazard function (also called the *hazard rate* or *intensity function*) is:

$$h(t) = \lim_{\Delta t \rightarrow 0} P(t \leq T < t + \Delta t \mid T \geq t) / \Delta t$$

Informally:  $h(t)$  is the instantaneous rate of departure per unit time, given the person is still present at time  $t$ . It has units of *events per hour*.

For office visits, the hazard is not constant: - Low in the first hour or two (the person just arrived) - Rising through the morning and accelerating after lunch - Peaking around the typical end-of-day window (4–6pm) - Dropping sharply for the rare very-long sessions

This increasing-then-plateau shape is why the estimated Weibull shape parameter  $\rho \approx 2.1$  (see Section 6) — a shape greater than 1 means the hazard grows with time.

## Cumulative Hazard and Its Relation to $S(t)$

The cumulative hazard is:

$$H(t) = \int_0^t h(s) \, ds$$

The fundamental identity linking  $H$ ,  $h$ , and  $S$  is:

$$S(t) = \exp(-H(t))$$

This means:  $-H(t) = -\log S(t)$   $-h(t) = -d/dt \log S(t)$

These relationships are used internally by `lifelines` when computing predictions. For example, `predict_median(X)` solves  $S(t | X) = 0.5$  for  $t$ , which is equivalent to solving  $H(t | X) = \log 2$ .

## The Likelihood with Censoring

When fitting any model, we need to specify the likelihood contribution of each observation. Let  $f(t) = h(t) \cdot S(t)$  be the density of the event time.

- **Complete session** (observed tap-out at  $t$ ): contributes  `$f(t) = h(t) \cdot S(t)$`  to the likelihood
- **Censored session** (no tap-out; still present at  $t^*$ ): contributes  `$S(t^*)$`  — the probability of not having left by the censoring time

The full log-likelihood is:

$$\ell(\theta) = \sum_i [\delta_i \log f(t_i; \theta) + (1-\delta_i) \log S(t_i; \theta)]$$

where  $\delta_i = 1$  for complete sessions and  $\delta_i = 0$  for censored sessions.

In the codebase, this is implemented via the `event_observed` column:

```
# model.py - the same pattern is used in both fit_cox and fit_weibull
df["event_observed"] = (~df["censored"]).astype(int)

# 1 = tap-out observed (complete)
# 0 = no tap-out (censored)
```

`lifelines` takes `duration_col` and `event_col` and constructs this likelihood internally. The `penalizer=0.01` adds an L2 regularisation term to prevent overfitting on the covariate coefficients.

---

## 5. Cox Proportional Hazards Model

---

### Model Specification

The Cox model specifies the hazard as:

$$h(t | X) = h_0(t) \cdot \exp(\beta^T X)$$

where: -  $h_0(t)$  is the **baseline hazard**: the hazard for a person with all covariates equal to zero. It is left completely unspecified (non-parametric). -  $\exp(\beta^T X)$  is a multiplicative covariate adjustment.

This is a **semi-parametric** model: parametric in the covariate part ( $\beta$  is a finite vector), non-parametric in the baseline hazard. The key assumption is *proportionality*: the hazard ratio between any two covariate profiles is constant over time.

$$h(t | X_a) / h(t | X_b) = \exp(\beta^T (X_a - X_b)) \quad \leftarrow \text{does not depend on } t$$

For office data this is an approximation: the relative departure rate between an early-morning arrival and a late-morning arrival may genuinely vary throughout the day. In practice, the assumption is usually good enough for prediction, though it should be tested with Schoenfeld residuals if inference on  $\beta$  is important.

### Covariate Interpretation

Each element of  $\beta$  gives a log hazard ratio:



| Covariate                 | If $\beta > 0$                              | If $\beta < 0$                        |
|---------------------------|---------------------------------------------|---------------------------------------|
| <code>arrival_hour</code> | Later arrivers leave sooner (higher hazard) | Later arrivers stay longer            |
| <code>day_of_week</code>  | Later in the week → shorter stays           | Earlier in the week → shorter stays   |
| <code>is_weekend</code>   | Weekend visitors have shorter stays         | Weekend visitors stay longer          |
| <code>person_freq</code>  | Frequent visitors tend to leave sooner      | Frequent visitors tend to stay longer |

The hazard ratio `exp( $\beta_k$ )` has a direct interpretation: a one-unit increase in covariate  $k$  multiplies the departure rate by `exp( $\beta_k$ )`, holding all other covariates constant.

## Partial Likelihood and Why `AIC_partial_`

Cox's key insight was that the baseline hazard  $h_0(t)$  cancels when you write the conditional probability that a specific person departs next, among all those still present at that time:

$$P(\text{person } i \text{ leaves at } t_i \mid \text{one departure from risk set } R_i) = \exp(\beta^T X_i) / \sum_{j \in R_i} \exp(\beta^T X_j)$$

Taking the product of these probabilities over all event times gives the *partial likelihood*, which depends only on  $\beta$  and the ordering of events, not on  $h_0$ . This means:

1. We can estimate  $\beta$  without ever specifying the baseline hazard shape.
2. The resulting information criterion is labelled `AIC_partial_` in `lifelines`: it is computed from the partial log-likelihood, not the full parametric likelihood. This makes it **not directly comparable** to the Weibull AIC (Section 8).

## Prediction

The Cox model prediction used in the pipeline is:

```
# model.py
return model.predict_median(X) # for CoxPHFitter
```

`predict_median` finds the time  $t$  at which  $S(t \mid X) = 0.5$ , using the Breslow estimator to reconstruct  $h_0(t)$  from the partial likelihood residuals. A limitation: if the covariate profile corresponds to a very long-stay individual, the survival curve may never reach 0.5 within the observed time range, making the median undefined. The Weibull AFT model (Section 6) avoids this.

---

## 6. Weibull Accelerated Failure Time Model

---

### AFT Formulation

An Accelerated Failure Time (AFT) model places a linear model directly on the log of the event time:

$$\log T = \mu + \beta^T X + \sigma \varepsilon$$

where  $\varepsilon$  follows a location-scale distribution. For the Weibull AFT,  $\varepsilon$  follows a Gumbel (extreme-value) distribution. The term "accelerated failure" comes from the covariate effect: if  $\beta^T X$  is negative,  $\exp(\beta^T X) < 1$ , and the visit duration is scaled down — the clock runs faster for that person.

Compare with the Cox model: Cox multiplies the *hazard* by a covariate factor; AFT multiplies the *time* by a factor. Both are valid models; they make different distributional assumptions.

The Weibull distribution belongs simultaneously to the AFT class and the PH class, making the Weibull AFT and Weibull PH models equivalent up to reparameterisation. This is unique to the Weibull.

### The Weibull Distribution

The Weibull survival function is:

$$S(t) = \exp(-(t/\lambda)^\rho)$$

with scale parameter  $\lambda > 0$  and shape parameter  $\rho > 0$ . The corresponding hazard is:

$$h(t) = (\rho/\lambda) \cdot (t/\lambda)^{(\rho-1)}$$

The shape parameter  $\rho$  controls the hazard trajectory:

| $\rho$           | Hazard over time          | Interpretation                                                   |
|------------------|---------------------------|------------------------------------------------------------------|
| $\rho < 1$       | Decreasing                | People who stay longer are <i>less</i> likely to leave (unusual) |
| $\rho = 1$       | Constant                  | Memoryless; exponential distribution                             |
| $\rho > 1$       | Increasing                | People who stay longer are <i>more</i> likely to leave soon      |
| $\rho \approx 2$ | $\approx$ Linear increase | Roughly Rayleigh-distributed durations                           |

In the monitoring data from this project,  $\rho$  ranges from **2.05 to 2.23** across all batches — strongly increasing hazard. This matches intuition: after the first few hours, the departure rate accelerates as people approach their planned end-of-day.

## Where $\rho$ Appears in the Codebase

`lifelines` fits  $\log(\rho)$  internally (log-scale parameterisation avoids a positivity constraint). The monitoring code recovers  $\rho$  as:

```
# monitoring.py
rho_log = float(model.params_["rho_"]["Intercept"])
weibull_rho = float(np.exp(rho_log))    # rho ≈ 2.05-2.23 in real data
```

## Prediction from Weibull AFT

The Weibull AFT gives a closed-form expression for the conditional expectation:

$$E[T \mid X] = \lambda(X) \cdot \Gamma(1 + 1/\rho)$$

where  $\lambda(X)$  is the estimated scale parameter as a function of covariates and  $\Gamma$  is the gamma function. This is what `predict_expectation` returns:

```
# model.py
return model.predict_expectation(X)    # for WeibullAFTFitter
```

This is why the Weibull AFT is the **default model** in the CLI: it gives a direct, closed-form expected duration for every session. Unlike `predict_median`, it never becomes undefined. It is also more interpretable for operational purposes ("this person will stay an average of 8.2 more hours") than a median survival time.

---

## 7. Feature Engineering

---

All four covariates are computed in `features.py:add_features()` from the `tap_in` timestamp and the session count per person.

### `arrival_hour`

The integer hour of the tap-in event (0–23). Captures the daily rhythm: someone arriving at 07:00 is likely planning a full day; someone arriving at 11:30 may leave at a similar absolute time (say, 17:00), giving a shorter session. The covariate enters the model linearly — a simple monotonic assumption that early arrivals stay longer.

### `day_of_week`

Integer 0 (Monday) through 6 (Sunday). Captures weekly patterns: Fridays typically have shorter sessions; Mondays sometimes longer. Like `arrival_hour`, it enters linearly, which assumes the day-of-week effect on log-hazard is monotone across the week. This is an approximation; one-hot encoding would be more flexible.

### `is_weekend`

Binary flag: 1 for Saturday and Sunday, 0 otherwise. The weekend office population is qualitatively different — typically fewer people, often focused deep work with different departure patterns. Keeping this as a separate flag rather than relying on `day_of_week = 5 or 6` gives it a direct coefficient in the hazard ratio.

### `person_freq`

The total count of sessions recorded for this person across the entire dataset. A proxy for regularity: high-frequency visitors (daily commuters) are likely to have more predictable and possibly shorter sessions (fixed routine), while infrequent visitors may stay longer (less familiar with the building, more ad-hoc agenda).

Note that `person_id` itself is deliberately excluded. Including it as a fixed effect would require one coefficient per person (high-cardinality), would not generalise to new individuals, and would overfit on small person-level samples. `person_freq` serves as a continuous, low-dimensional summary of regularity.

## Penalisation

Both the Cox and Weibull models are fitted with `penalizer=0.01` :

```
# model.py
cph = CoxPHFitter(penalizer=0.01)
aft = WeibullAFTFitter(penalizer=0.01)
```

This adds an L2 (ridge) penalty:  $-0.01 \cdot \|\beta\|^2$  to the log-likelihood. The effect is to shrink all coefficients toward zero, reducing variance at the cost of a small bias. With moderate-sized datasets (hundreds to thousands of sessions) and only four covariates, the penalty is mild but stabilises fitting when `person_freq` has large variance.

---

## 8. Model Fitting, Likelihood, and AIC

### The Log-Likelihood with Censoring

Substituting the Weibull density  $f(t) = (p/\lambda)(t/\lambda)^{(p-1)} \cdot \exp(-(t/\lambda)^p)$  and survival function  $S(t) = \exp(-(t/\lambda)^p)$  into the general likelihood formula:

For a **complete** session at duration  $t$ :

$$\log f(t) = \log p - \log \lambda + (p-1) \log(t/\lambda) - (t/\lambda)^p$$

For a **censored** session with last-known duration  $t^*$ :

$$\log S(t^*) = -(t^*/\lambda)^p$$

The total penalised log-likelihood maximised by `WeibullAFTFitter` is:

$$\ell_{\text{pen}}(\theta) = \sum_i [\delta_i \log f(t_i; \theta) + (1-\delta_i) \log S(t_i; \theta)] - p/2 \cdot \|\beta\|^2$$

where  $p = 0.01$ . The `event_observed` column (0 for censored, 1 for complete) is the  $\delta_i$  indicator.

### Akaike Information Criterion

The AIC balances fit and complexity:

$$\text{AIC} = -2 \cdot \ell + 2k$$

where  $\ell$  is the maximised log-likelihood and  $k$  is the number of free parameters.

**Important caveat on interpretation over time:** In the monitoring dashboard, AIC grows monotonically with batch size (from 2,277 at Batch 1 to 14,068 at Batch 6). This is expected and correct: more observations yield a larger magnitude log-likelihood, which increases AIC proportionally. **Use AIC for model selection** (comparing the Cox model to the Weibull model on the *same* dataset), not for tracking performance as the dataset grows.

For the Cox model, `lifelines` reports `AIC_partial_` computed from the partial log-likelihood. This is **not directly comparable** to the Weibull `AIC_`, because the Cox partial likelihood marginalises over the baseline hazard rather than fully specifying it.

## 9. Model Evaluation and Monitoring

### 9.1 Concordance Index (C-index)

The C-index (Harrell's c-statistic) measures **discrimination**: how well the model ranks sessions by duration.

**Definition:** Among all pairs of sessions where both durations are comparable (one complete session, or two complete sessions), the C-index is the fraction of pairs where the model assigns a higher predicted duration to the session that actually lasted longer.

$$C = P(\text{predicted}_i > \text{predicted}_j \mid \text{observed}_i > \text{observed}_j)$$

Pairs where one member is censored before the other's event time are excluded (the ordering is ambiguous). This is the correct handling — not an approximation.

**Interpretation:** -  $C = 0.5$ : random ordering (no better than a coin flip) -  $C = 1.0$ : perfect discrimination -  $C = 0.7$ – $0.8$ : considered good for biological survival data -  $C < 0.6$ : weak discrimination; covariates explain little variance in duration ordering

In the monitoring data, C-index ranges from **0.51 to 0.56** across batches. This is modest, which is expected: individual departure times are highly stochastic, and the four covariates explain a limited but genuine fraction of the variance. The dashboard shows  $C = 0.5$  as a dashed reference line.

`lifelines` computes the C-index as `model.concordance_index_` for both Cox and Weibull fitters.

## 9.2 Predictive Accuracy on Complete Sessions

For censored sessions we do not observe the true duration, so we cannot compute prediction residuals. Predictive metrics are therefore restricted to the `~sessions["censored"]` subset.

Let  $\hat{y}_i$  = predicted duration and  $y_i$  = observed duration for complete sessions. The residual is  $r_i = \hat{y}_i - y_i$ .

| Metric             | Formula                            | Typical value (this data) | Interpretation                           |
|--------------------|------------------------------------|---------------------------|------------------------------------------|
| <b>MAE</b>         | $\text{mean}( r_i )$               | 2.37–2.65h                | Average absolute error in hours          |
| <b>RMSE</b>        | $\sqrt{\text{mean}(r_i^2)}$        | 2.84–3.14h                | Penalises large errors more than MAE     |
| <b>Bias</b>        | $\text{mean}(r_i)$                 | +2.15 to +2.40h           | Positive = systematic over-prediction    |
| <b>MAPE</b>        | $\text{mean}( r_i /y_i) \cdot 100$ | 38–44%                    | Scale-free; inflated when $y_i$ is small |
| <b>Correlation</b> | $\text{corr}(\hat{y}_i, y_i)$      | 0.02–0.14                 | Weak linear relationship                 |

**On the positive bias:** The model consistently over-predicts for complete sessions (bias  $\approx +2.2$ h). This is largely a consequence of the Weibull `predict_expectation` returning  $E[T | X]$  over the full distribution, while very long sessions (many of which are censored and therefore not in the evaluation set) pull the predicted mean upward. The complete sessions available for evaluation under-represent the right tail. This is not a model bug but a known consequence of evaluating on a non-random subset.

## 9.3 Censoring Rate as a Data Pipeline Signal

The `censoring_rate` ( $\approx 19$ – $21\%$  in the monitoring data) should be tracked independently of model performance metrics. A sudden spike — say from  $20\%$  to  $40\%$  — almost certainly indicates a hardware or software issue in the access control system (a broken reader, a network outage causing lost events), not a genuine change in human behaviour. The "Censoring Rate over Time" panel in the dashboard is a data quality health check, not a model quality metric.

## 9.4 Distribution Shift Detection

The monitoring simulation introduces gradual drift: each batch shifts mean arrival time by +0.1h (later) and mean duration by -0.15h (shorter). This is visible in the `mean_observed` column:

```
Batch 1: mean_observed = 7.68h
Batch 2: mean_observed = 7.62h
Batch 3: mean_observed = 7.57h
...
Batch 6: mean_observed = 7.38h
```

A well-monitored pipeline would detect this via the "Mean Duration: Observed vs Predicted" chart and trigger a model retrain. The `--rolling` flag in `generate_data.py` fits each batch independently (rolling window), making the model faster to react to drift at the cost of higher metric variance. The default cumulative training mode absorbs drift slowly but produces more stable estimates.

---

## 10. The Production Pipeline

### End-to-End Workflow

```
Raw CSV → build_sessions → add_features → fit_weibull → predict_durations → results
```

#### Step 1: Generate or supply data

```
# Generate synthetic test data
python -m office_duration.generate_data \
    --people 50 --days 30 --tailgate-rate 0.2 \
    --output data/taps.csv
```

#### Step 2: Run the pipeline



```
python -m office_duration.cli data/taps.csv \
  --output results.csv \
  --model weibull \           # or cox
  --max-duration 16.0 \
  --monitor \                 # save metrics to JSONL
  --run-label "2025-W03" \    # human-readable run label
  --dashboard                  # generate HTML dashboard
```

The output `results.csv` has one row per session:

| Column                       | Description                                   |
|------------------------------|-----------------------------------------------|
| <code>person_id</code>       | Individual identifier                         |
| <code>tap_in</code>          | Entry time                                    |
| <code>tap_out</code>         | Exit time, or NaT if censored                 |
| <code>censored</code>        | True/False                                    |
| <code>observed_hours</code>  | True duration (complete sessions only)        |
| <code>estimated_hours</code> | Model's predicted $E[T   X]$ for all sessions |

## Monitoring Simulation

To populate the dashboard with multi-batch history without real data:

```
python -m office_duration.generate_data \
  --simulate-monitoring \
  --batches 8 \
  --batch-days 14 \
  --people 50 \
  --tailgate-rate 0.2 \
  --drift \           # enable gradual distribution shift
  --model weibull \
  --dashboard-path monitoring/dashboard.html
```

This runs the full pipeline on 8 sequential 14-day batches, appends metrics to `monitoring/metrics.jsonl`, and generates `monitoring/dashboard.html`. Use `--rolling` to fit each batch independently (rolling window) rather than cumulatively.

---

## 11. Limitations and Extensions

---

### Testing the Proportionality Assumption

The Cox model's core assumption (constant hazard ratios over time) should be tested with **Schoenfeld residuals**: if a covariate's Schoenfeld residual shows a trend over time, the proportionality assumption is violated for that covariate. The `lifelines` library provides `CoxPHFitter.check_assumptions()` which performs this test automatically.

### Frailty and Random Effects

`person_freq` is a crude individual-level adjustment. A more principled approach is a **frailty model**: a Cox or Weibull model with a random multiplicative term per person (Gamma or log-normal frailty). This is equivalent to a mixed-effects survival model. `lifelines` supports this via `CoxPHFitter(baseline_estimation_method="breslow")` with shared frailty, or through `WeibullAFTFitter` with a formula string including a random effect.

### Multiple Entries per Day

The current model treats each tap-in as the start of an independent session. In reality, some people enter, exit for lunch, and return. These within-day sessions are statistically correlated (a morning and afternoon session for the same person on the same day are not independent). A clustered survival model or multi-state model would handle this correctly.

### Investigating the Positive Bias

The consistent +2.2h positive bias on complete sessions deserves further investigation. Comparing `predict_expectation(X)` against `predict_median(X)` on the same uncensored subset would clarify whether the Weibull distribution's tail is too heavy for this dataset. If the median prediction is closer to the observed mean, a median-based prediction strategy may be more appropriate for operational reporting.

## The Arbitrary 16-Hour Cap

`max_duration_hours=16` is the default censoring cap. The correct value should be informed by building operating hours and the longest plausible legitimate session. A cap that is too low compresses the right tail of the censored duration distribution, biasing the likelihood; a cap that is too high adds noise. Sensitivity analysis across cap values (12h, 14h, 16h, 18h) is recommended for production deployments.

---

## 12. Quick Reference

### Key Metrics

| Metric            | Formula                                                | <code>lifelines</code> attribute       | Interpretation                         |
|-------------------|--------------------------------------------------------|----------------------------------------|----------------------------------------|
| Concordance index | P(rank correct)                                        | <code>model.concordance_index_</code>  | 0.5 = random; higher is better         |
| Log-likelihood    | $\sum \delta_i \log f(t_i) + (1-\delta_i) \log S(t_i)$ | <code>model.log_likelihood_</code>     | Higher (less negative) = better fit    |
| AIC (Weibull)     | $-2 \cdot LL + 2k$                                     | <code>model.AIC_</code>                | Lower = better; compare same dataset   |
| AIC (Cox)         | $-2 \cdot \text{partial LL} + 2k$                      | <code>model.AIC_partial_</code>        | Not comparable to Weibull AIC          |
| Weibull $\rho$    | $\exp(\text{params\_}["\rho\_"] ["Intercept"])$        | computed in <code>monitoring.py</code> | $>1$ = increasing hazard               |
| MAE               | mean( $ \hat{y} - y $ ) in hours                       | computed in <code>monitoring.py</code> | Lower = better; complete sessions only |
| RMSE              | $\sqrt{\text{mean}((\hat{y} - y)^2)}$ in hours         | computed in <code>monitoring.py</code> | Penalises large errors                 |
| Bias              | mean( $\hat{y} - y$ ) in hours                         | computed in <code>monitoring.py</code> | Near 0 ideal; +ve = over-prediction    |
| MAPE              | mean( $ \hat{y} - y /y$ ) $\cdot 100\%$                | computed in <code>monitoring.py</code> | Scale-free error; sensitive to small y |
| Censoring rate    | $n_{\text{censored}} / n_{\text{total}}$               | computed in <code>monitoring.py</code> | Data quality signal                    |

## Model Comparison at a Glance

| Property                                   | Cox PH                                             | Weibull AFT                                               |
|--------------------------------------------|----------------------------------------------------|-----------------------------------------------------------|
| Parametric?                                | Semi-parametric                                    | Fully parametric                                          |
| Prediction output                          | Median survival<br>( <code>predict_median</code> ) | Expected duration<br>( <code>predict_expectation</code> ) |
| Handles covariate profiles with long tails | May return undefined median                        | Always returns a finite $E[T X]$                          |
| AIC comparable across model types?         | No ( <code>AIC_partial_</code> )                   | Yes ( <code>AIC_</code> )                                 |
| Requires distributional assumption?        | No                                                 | Yes (Weibull)                                             |
| Default in this codebase?                  | No                                                 | <b>Yes</b>                                                |

## CLI Flags Reference

| Flag                          | Default                                | Description                                     |
|-------------------------------|----------------------------------------|-------------------------------------------------|
| <code>--model</code>          | <code>weibull</code>                   | Choose <code>weibull</code> or <code>cox</code> |
| <code>--max-duration</code>   | <code>16.0</code>                      | Censoring cap in hours                          |
| <code>--monitor</code>        | <code>off</code>                       | Append metrics to JSONL after fitting           |
| <code>--run-label</code>      | (timestamp)                            | Human-readable label for this run               |
| <code>--metrics-path</code>   | <code>monitoring/metrics.jsonl</code>  | Metrics file path                               |
| <code>--dashboard</code>      | <code>off</code>                       | Generate HTML monitoring dashboard              |
| <code>--dashboard-path</code> | <code>monitoring/dashboard.html</code> | Dashboard output path                           |

## 13. Further Reading

**Lifelines library** Davidson-Pilon, C. et al. (2019). *lifelines: survival analysis in Python*. <https://lifelines.readthedocs.io>

**Cox model — original paper** Cox, D.R. (1972). Regression models and life-tables. *Journal of the Royal Statistical Society, Series B*, 34(2), 187–220.

**Weibull AFT and parametric survival models** Klein, J.P. & Moeschberger, M.L. (2003). *Survival Analysis: Techniques for Censored and Truncated Data* (2nd ed.). Springer.

**Concordance index** Harrell, F.E. (2015). *Regression Modeling Strategies* (2nd ed.), Chapter 17. Springer. Introduces the C-statistic and its relationship to the Wilcoxon statistic.

**Censoring and survival bias** Hernán, M.A. (2010). The hazards of hazard ratios. *Epidemiology*, 21(1), 13–15. A concise treatment of the interpretation pitfalls in survival analysis.

---

Document generated from the `office-duration-model` repository. Model fit statistics cited throughout ( $\rho$ , C-index, MAE, bias, censoring rate) are from `monitoring/metrics.jsonl` produced by a 6-batch cumulative Weibull AFT simulation with 50 people, 14-day batches, and 20% tailgate rate.